



Welcome to A Catholic Introduction to Artificial Intelligence Class 4! Machine Learning and Neural Networks

"We are entering a new world in which data may be more important than software." — Tim O'Reilly

Learning Objectives

By the end of this class, you will be able to define machine learning, explain how it differs from traditional programming, describe supervised and unsupervised learning, understand the basic idea of a neural network. At the heart of these concepts is the role of data and algorithms in achieving AI objectives.

1. Class 3 Review

Artificial intelligence began with great optimism in the 1950s and 1960s, as early researchers believed machines would soon be able to match human reasoning. Important figures like Alan Turing and John McCarthy helped lay the foundation, and early programs such as ELIZA and the General Problem Solver demonstrated that computers could mimic certain aspects of thinking. However, these systems worked only in limited situations and could not handle the complexity of the real world.

This gap between expectations and reality led to periods known as **AI Winters**, when funding and interest declined because AI systems failed to deliver on bold promises. As described in your lecture, these cycles were driven by **overpromising, technical limitations,**

and systems that worked in controlled demos but failed in practical use. A similar pattern occurred again in the 1980s with expert systems, which were useful but ultimately too rigid, expensive, and difficult to maintain.

Despite these setbacks, AI continued to evolve. Neural networks and machine learning introduced new ways for computers to learn from data rather than relying solely on fixed rules. This eventually led to modern advances like deep learning, which uses multiple layers to recognize complex patterns.

The lesson also highlights two important types of AI used today. **Discriminative AI** classifies data into categories, such as identifying whether an email is spam. **Generative AI**, on the other hand, creates new content, such as generating text or images. Understanding this distinction helps explain many modern AI tools.

Finally, AI has had a strong influence on culture and media. Films and stories have shaped how people think about AI—sometimes creating excitement, and other times fear—often exaggerating what AI can actually do.

Overall, the history of AI shows a repeating pattern: excitement, disappointment, and progress. Today's AI is more successful because of better data, stronger computing power, and improved techniques, but the lessons from the past remind us to stay realistic about what AI can achieve.

2. What Exactly is Data?

Data is, quite simply, information. It is the text messages you send, the pictures you snap on your phone, the high scores you rack up in your favorite video game, the songs you stream, and even the daily temperature outside. Data is a digital record of the world around us.

Now, why does this matter for artificial intelligence, and specifically, for a field called machine learning?

Think about how you learned to recognize a dog when you were a toddler. Your parents didn't sit you down with a rigid, technical checklist saying, "A dog is an entity possessing exactly four legs, a tail, fur, and an auditory output known as a bark." If they had, you might have easily confused a dog with a rather noisy cat! Instead, they pointed to different dogs in the park, in books, and on TV, saying, "Look, a dog!" After seeing dozens of different examples your brain naturally figured out the underlying pattern. You learned from experience.

Machine learning works in exactly the same way. Traditional computer programming is like giving a computer a strict recipe: "Do step A, then step B, and you get C." Machine learning, however, is about giving the computer a massive stack of digital flashcards and letting it figure out the recipe all by itself. Those flashcards? That is our data.

If we want to teach an AI program to recognize pictures of cats, we do not write a thousand lines of code mathematically describing pointy ears and whiskers. Instead, we feed the machine learning model thousands of images of cats (and probably some pictures of dogs and muffins, just so it learns the difference). The algorithm analyzes this data, looking for shapes, colors, and pixel patterns. The more high-quality data we provide, the smarter and more accurate the AI becomes.

But here is the catch, and it is a tremendously important one: an AI is only as good as the data it learns from. In the computer science world, we have a famous saying: “Garbage In, Garbage Out.” If you try to study for your calculus test using an ancient history textbook, you are not going to do very well, right?

Similarly, if we feed our AI biased, incomplete, or incorrect data, it will make terrible decisions. If we only show our cat-recognizing AI pictures of orange tabbies, it might confidently declare that a black cat is not a cat at all. Imagine an AI designed to screen resumes for a job, but it was only trained on data from successful applicants who all went to the exact same university. It would unfairly reject brilliant candidates from other schools simply because it had never seen them in its data before. This is why having diverse, accurate data isn’t just a technical requirement—it is a deeply ethical one.

Data is the fuel that powers the spectacular engine of artificial intelligence. It is the life experience that gives machine learning its “smarts.” As we move forward in this class, we will not just learn how these algorithms work; we will learn how to be responsible, thoughtful collectors and curators of data. Understanding data is your very first superpower in mastering AI. Get ready, because we are going to build some incredible things together!

What Data Means to AI

Data is the set of examples or observations that a model uses to learn. Each example can include:

- Features: the pieces of information used to describe the example, such as age, height, or pixel values in an image.
- Labels or targets: the answer the model should predict, like whether an email is spam, the species of a flower, or the price of a house.

When we say a model learns from data, we mean it looks for relationships between features and labels. For instance, if many pictures of cats and dogs are labeled correctly, the model tries to identify the patterns that distinguish a cat from a dog.

Why Data Quality Matters

Data quality is as important as the model. Good data is:

- Accurate

- Representative of real situations
- Clean and consistent

Poor data can lead to poor learning (i.e. the inability to find patterns). If the examples contain errors, bias, or missing values, the model may learn the wrong patterns. For a classroom, this is a great moment to discuss how real-world data often needs cleaning before it can teach a model well.

Let's look at an example.

```
In [1]: # Lets install pandas first; The pandas library is a powerful tool for data manipul
%pip install pandas
```

```
Requirement already satisfied: pandas in d:\career\contracting\hsc\catholic-intro-to-ai\venv\lib\site-packages (3.0.3)
Requirement already satisfied: numpy>=1.26.0 in d:\career\contracting\hsc\catholic-intro-to-ai\venv\lib\site-packages (from pandas) (2.4.5)
Requirement already satisfied: python-dateutil>=2.8.2 in d:\career\contracting\hsc\catholic-intro-to-ai\venv\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in d:\career\contracting\hsc\catholic-intro-to-ai\venv\lib\site-packages (from pandas) (2026.2)
Requirement already satisfied: six>=1.5 in d:\career\contracting\hsc\catholic-intro-to-ai\venv\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

```
[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
In [2]: # Load the dataset from Local CSV file. We have a synthetic dataset of homework as
# stored in a CSV file, and we will use the pandas library to read it into a DataFrame
import pandas as pd
hw_df = pd.read_csv('synthetic_homework_dataset.csv')
```

```
In [3]: # Let's take a look at the first few rows of the dataset.
hw_df.head()
```

```
Out[3]:
```

	student_id	assignment_id	assignment_type	num_questions	difficulty	days_until_due
0	S140	A8	reading	19	4	3
1	S117	A16	worksheet	5	5	4
2	S147	A7	quiz_prep	20	4	6
3	S137	A28	reading	8	2	1
4	S105	A14	worksheet	12	4	3

```
In [4]: # Now, Let's take a look at the last few rows of the dataset
hw_df.tail()
```

Out[4]:

	student_id	assignment_id	assignment_type	num_questions	difficulty	days_until_due
495	S114	A29	coding	22	4	4
496	S101	A27	reading	18	5	4
497	S132	A24	worksheet	5	1	1
498	S105	A24	worksheet	14	2	6
499	S120	A7	coding	6	3	4

In [5]: *# Finally, let's get some basic information about the dataset, such as the number of missing values.*
 hw_df.info()

```
<class 'pandas.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   student_id                            500 non-null    str
1   assignment_id                          500 non-null    str
2   assignment_type                        500 non-null    str
3   num_questions                         500 non-null    int64
4   difficulty                             500 non-null    int64
5   days_until_due                        500 non-null    int64
6   date_assigned                         500 non-null    str
7   date_submitted                        500 non-null    str
8   time_spent_minutes                    500 non-null    float64
9   num_work_sessions                     500 non-null    int64
10  completed_on_time                      500 non-null    int64
11  submitted_late_days                    500 non-null    int64
12  prior_completion_rate                  500 non-null    float64
13  prior_avg_grade                        500 non-null    float64
14  prior_avg_homework_time                500 non-null    float64
dtypes: float64(4), int64(6), str(5)
memory usage: 58.7 KB
```

In [6]: *# Let's look at the dataset shape. A dataset shape is a tuple that represents the number of rows and the second element is the number of columns.*
 hw_df.shape

Out[6]: (500, 15)

Observation(s):

- We have 500 rows and 15 columns of data.

In [7]: *# Now, let's get some basic statistics about the dataset, such as mean, median, standard deviation, etc.*
 hw_df.describe(include='all').T

Out[7]:

	count	unique	top	freq	mean	std	min	25%
student_id	500	51	S140	19	NaN	NaN	NaN	NaN
assignment_id	500	40	A28	19	NaN	NaN	NaN	NaN
assignment_type	500	5	reading	106	NaN	NaN	NaN	NaN
num_questions	500.0	NaN	NaN	NaN	14.894	5.798449	5.0	10.0
difficulty	500.0	NaN	NaN	NaN	3.054	1.429395	1.0	2.0
days_until_due	500.0	NaN	NaN	NaN	3.544	1.755628	1.0	2.0
date_assigned	500	60	2026-02-18	14	NaN	NaN	NaN	NaN
date_submitted	500	65	2026-02-13	18	NaN	NaN	NaN	NaN
time_spent_minutes	500.0	NaN	NaN	NaN	58.2416	24.730649	8.9	40.175
num_work_sessions	500.0	NaN	NaN	NaN	1.664	0.774793	1.0	1.0
completed_on_time	500.0	NaN	NaN	NaN	0.858	0.3494	0.0	1.0
submitted_late_days	500.0	NaN	NaN	NaN	0.382	1.036448	0.0	0.0
prior_completion_rate	500.0	NaN	NaN	NaN	0.84774	0.099648	0.52	0.78
prior_avg_grade	500.0	NaN	NaN	NaN	84.8058	8.911771	60.0	79.0
prior_avg_homework_time	500.0	NaN	NaN	NaN	40.5502	10.027218	11.3	33.7

Observation(s):

- The dataset contains **500 assignment records**.
- There are **51 unique students** and **40 unique assignments** represented in the data.
- The most common student ID is **S140**, appearing **19 times**.
- The most common assignment ID is **A28**, also appearing **19 times**.
- There are **5 different assignment types**, with **reading assignments** being the most common (**106 occurrences**).
- Assignments contain an average of about **15 questions**.
 - Minimum: 5 questions
 - Maximum: 24 questions
- Assignment difficulty is centered around **moderate difficulty**.
 - Mean difficulty: about 3.05 on a 1–5 scale

- Median difficulty: 3
- Students are typically given about **3–4 days to complete assignments**.
 - Minimum: 1 day
 - Maximum: 6 days
- The dataset spans **60 different assignment dates** and **65 different submission dates**.
- Students spend an average of about **58 minutes per assignment**.
 - Median: about 55.5 minutes
 - Maximum observed time: 145.9 minutes
- Most assignments are completed in **a single work session**.
 - Mean sessions: 1.66
 - Median sessions: 1
 - Maximum sessions: 5
- The on-time completion rate is very high.
 - Mean value of `completed_on_time` : 0.858
 - This suggests about **85.8% of assignments were submitted on time**.
- Late submissions appear relatively uncommon.
 - Median late days: 0
 - Average late days: only 0.382
 - Maximum lateness: 4 days
- Students generally have strong historical performance.
 - Average prior completion rate: about 84.8%
 - Median prior completion rate: 85%
- Prior grades are relatively high overall.
 - Average prior grade: about 84.8%
 - Median prior grade: 85%
 - Maximum prior grade: 100%
- Students previously spent less time on homework than the current assignments require.
 - Average prior homework time: about 40.6 minutes
 - Current average assignment time: about 58.2 minutes
- The standard deviations suggest moderate variation in:
 - time spent,
 - prior grades,
 - and number of questions.
- Several variables show possible skewness:
 - `submitted_late_days` is heavily skewed toward zero.

- `num_work_sessions` is also concentrated around 1 session.
- No missing numerical data is visible in the summary statistics since all numeric columns report counts of 500.

3. Algorithms

An algorithm is a step-by-step set of instructions designed to solve a problem or complete a task.

Think of it like a recipe for baking your favorite chocolate chip cookies. If you want to end up with delicious cookies, you cannot just throw flour, eggs, and chocolate into a cold oven and hope for the best. You follow a specific sequence:

1. Preheat the oven to 350 degrees.
2. Mix the dry ingredients in one bowl.
3. Mix the wet ingredients in another.
4. Combine them.
5. Bake for 10 minutes.

That recipe is an algorithm. Your morning routine—waking up, brushing your teeth, packing your backpack, and walking to the bus stop—is an algorithm.

In the world of computer science, algorithms are incredibly important because computers are fundamentally not very smart. They are astonishingly fast, and they have perfect memories, but they are incredibly dumb. They will do exactly what you tell them to do, nothing more and nothing less.

In traditional computer programming, a human programmer has to write out every single step of the algorithm by hand. If we want a character in a video game to jump when you press the spacebar, a programmer wrote an algorithm to make that happen exactly the same way every time.

In machine learning, we do not write a step-by-step algorithm that says "Look for two pointy ears and whiskers." Instead, we write a special kind of algorithm—a *learning* algorithm. This set of instructions tells the computer, "Here is a massive pile of data. Analyze it, find the patterns, and figure out the rules for yourself."

The AI takes our learning algorithm, looks at all that data, and essentially writes its *own* incredibly complex recipe for recognizing a cat.

You use AI algorithms constantly. When TikTok or YouTube recommends a video it knows you will laugh at, that is a recommendation algorithm looking at your past viewing data and predicting what you want to see next. When Google Maps recalculates your route to avoid a traffic jam, a routing algorithm is crunching speed and distance data to find the fastest path.

4. Learning From Data - The heart of Machine Learning

Machine learning is a way for computers to learn patterns from data instead of being explicitly programmed with rules. Contrast this with traditional programming: in traditional systems, humans write rules; in machine learning, the system learns those rules from examples. Use a simple example like spam filtering. Instead of writing rules for every possible spam email, the system studies many examples and learns patterns that distinguish spam from non-spam.

Examples and Applications

Here are some everyday examples:

- Spam filters learn from emails labeled spam or not spam.
- Recommendation systems learn from what people watch or buy.
- Image recognition learns from labeled photos.
- Language models learn from text and can generate writing or answer questions.

Each example shows how data turns into a model that can make predictions or decisions.

The Learning Process

A typical learning process includes:

1. Collecting data
2. Preparing and cleaning the data
3. Choosing a model type
4. Training the model on data
5. Evaluating performance
6. Improving the model with better data or techniques

This mirrors scientific investigation: make observations, form a hypothesis, test it, and refine it.

Training, Validation, and Testing

A key idea is dividing data into separate sets:

- **Training data** : used to teach the model.
- **Validation data** : used to tune the model and choose the best version.
- **Test data** : used to evaluate how well the model works on new, unseen examples.

This is like giving students practice questions, review quizzes, and a final exam. If a model does well only on its training data but badly on test data, it is **overfitting** : it memorized

the examples instead of learning general rules.

Models and Learning Algorithms

A model is a mathematical function that maps input data to an output prediction. Different learning algorithms build different kinds of models. Some examples:

- **Linear regression** : learns a straight-line relationship between input and output.
- **Decision trees** : split data into branches based on feature values.
- **Random Forests** : combine many decision trees trained on different random subsets of the data and average their predictions.
- **Neural networks** : combine many layers of math operations to learn complex patterns.

The learning algorithm adjusts the model's internal settings—often called parameters—so the model's predictions match the training data as closely as possible.

5. Supervised Learning

Supervised learning is the most common type of machine learning in the world today. To understand supervised learning, think about how you study for a vocabulary quiz using flashcards. On the front of the card, you have the word (let's say, *algorithm*). On the back of the card, you have the definition. When you study, you look at the front, make your best guess, and then flip the card over to check if you were right. If you were wrong, you adjust your understanding and try again next time. You are learning because you have the correct answers right there to guide you.

Supervised learning works exactly the same way. We feed our AI algorithm a massive dataset where every single piece of information already has the "correct answer" attached to it. In computer science, we call these correct answers **labels**.

Let's look at one of the most famous examples of supervised learning: the spam filter in your email.

If we want to build an AI that catches spam, we start by gathering tens of thousands of past emails. Crucially, humans have already gone through this data and attached a label to every single one. Email A gets the label "Spam." Email B gets the label "Not Spam."

During the training phase, we give the algorithm the text of Email A and ask it to guess what it is. At first, the computer is just making random guesses. Let's say it guesses "Not Spam." The algorithm then "flips the flashcard over," checks the label, and realizes it was wrong. It then adjusts its internal math, noting that emails containing phrases like "CONGRATULATIONS YOU WON" in all capital letters are highly likely to be spam.

It does this over and over, millions of times, constantly tweaking its own rules based on whether its guess matched the correct label. The labeled data is *supervising* the learning process.

Whether it is a doctor using AI to identify whether an X-ray shows a "fracture" or "no fracture," or an app identifying whether the song playing in the coffee shop is "Taylor Swift" or "The Beatles," supervised learning is behind it. You give the computer the examples, you give it the answers, and you let the algorithm connect the dots!

A Small Example: Predicting On-Time Completion

Let's put supervised learning into practice using the same homework dataset from Section 2. We will give the model a set of *features* (`prior_completion_rate` , `time_spent_minutes` , `difficulty`) and the correct *label* (`completed_on_time`) for a set of training assignments, then check how well it predicts the label for assignments it has never seen before — just like flipping the flashcard over to check the answer.

A *decision tree classifier* is a supervised learning algorithm that makes predictions by asking a series of yes-or-no questions about the input data, arranged into a branching, tree-like structure. Starting at the root, the model picks the feature and threshold that best splits the training data into more "pure" groups — for example, asking "is `prior_completion_rate` above 0.75?" — then repeats this process on each resulting branch, asking further questions until it reaches a leaf node that assigns a final class label. This mirrors how a person might work through a flowchart: each internal node is a test on a single feature, each branch is the outcome of that test, and each leaf is a decision. Decision trees are popular because they're easy to interpret and visualize (you can literally draw the tree and see why it made a prediction), require little data preprocessing, and can capture nonlinear relationships that a straight-line model like linear regression would miss. Their main weakness is a tendency to overfit — a tree grown too deep can carve out a rule for nearly every training example, memorizing noise instead of learning general patterns — which is why parameters like `max_depth` are used to limit how many questions the tree is allowed to ask, and why ensemble methods like random forests (which train many trees on different random subsets of the data and average their votes) are often preferred in practice for better accuracy and stability.

```
In [8]: # A small supervised learning example: predicting whether an assignment was complet
%pip install scikit-learn

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Features (X) and the correct answers, or "labels" (y)
features = ["prior_completion_rate", "time_spent_minutes", "difficulty"]
X = hw_df[features]
y = hw_df["completed_on_time"]
```

```

# Split the labeled data into a training set and a test set,
# just like the training/validation/test idea from Section 4
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train ("supervise") the model using only the training labels
model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)

# Now test the model on assignments it has never seen before
predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)

print(f"The model correctly predicted on-time completion {accuracy:.0%} of the time")

```

```

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
Requirement already satisfied: scikit-learn in d:\career\contracting\hsc\catholic-intro-to-ai\venv\lib\site-packages (1.9.0)
Requirement already satisfied: numpy>=1.24.1 in d:\career\contracting\hsc\catholic-intro-to-ai\venv\lib\site-packages (from scikit-learn) (2.4.5)
Requirement already satisfied: scipy>=1.10.0 in d:\career\contracting\hsc\catholic-intro-to-ai\venv\lib\site-packages (from scikit-learn) (1.17.1)
Requirement already satisfied: joblib>=1.4.0 in d:\career\contracting\hsc\catholic-intro-to-ai\venv\lib\site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: narwhals>=2.0.1 in d:\career\contracting\hsc\catholic-intro-to-ai\venv\lib\site-packages (from scikit-learn) (2.22.1)
Requirement already satisfied: threadpoolctl>=3.5.0 in d:\career\contracting\hsc\catholic-intro-to-ai\venv\lib\site-packages (from scikit-learn) (3.6.0)
Note: you may need to restart the kernel to use updated packages.
The model correctly predicted on-time completion 81% of the time on data it had never seen before.

```

Observation(s):

- The model was never told any *rules* for what counts as "likely to be late." It only saw examples — features paired with the correct label — and worked out its own rules from the patterns in the data.
- This is the same "flashcard" process described above, just running thousands of times inside the algorithm instead of in a student's head.

6. Unsupervised Learning

While supervised learning relies on meticulously labeled datasets, unsupervised learning involves providing the algorithm with raw, unlabeled data. In this scenario, the objective is not to predict a known, predefined outcome, but rather to discover inherent structures, patterns, or groupings hidden within the dataset itself. The system is tasked with finding order in a chaotic environment without human guidance.

A primary application of unsupervised learning is *clustering*. Consider a large dataset containing the purchasing histories of thousands of retail customers. In a supervised model,

a data scientist might label these customers to predict a specific outcome, such as "likely to respond to a discount." In an unsupervised model, however, the algorithm simply ingests the raw data and groups the customers based on identified similarities. It might cluster individuals into distinct demographic profiles or purchasing habits that a human analyst had not previously recognized or defined.

Another critical application is *anomaly detection*. Because an unsupervised algorithm becomes highly proficient at understanding the baseline structure of a dataset, it is exceptionally good at flagging data points that deviate from the norm. This is the underlying technology utilized by financial institutions to detect credit card fraud; the algorithm recognizes an unusual purchasing pattern—an anomaly—and flags the transaction for review.

By allowing the system to derive its own classifications rather than strictly guiding its training, unsupervised learning enables us to uncover complex insights and data relationships that might otherwise remain obscured.

This conceptual framework should serve as a strong, rigorous addition to your Introduction to AI curriculum. Please proceed whenever you are ready for the next topic.

A Small Example: Finding Hidden Groups

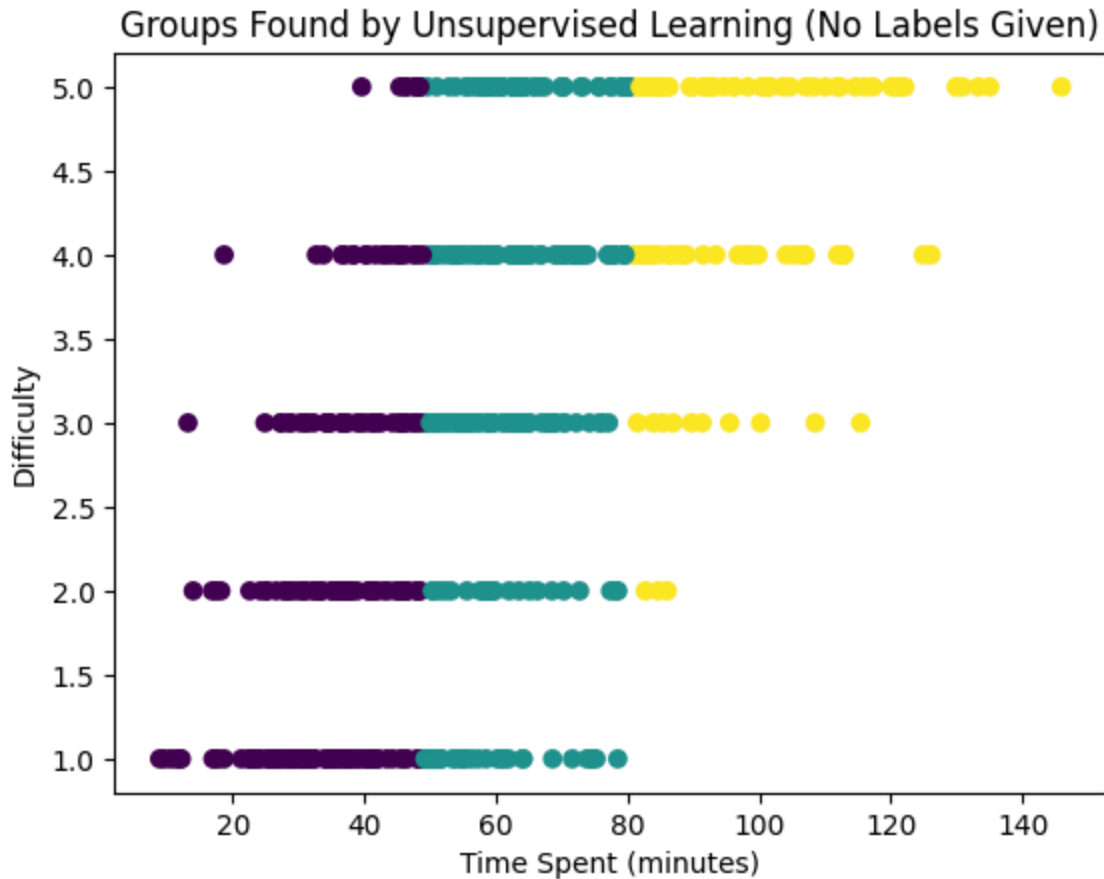
Now let's try the opposite. This time we will not give the algorithm any labels at all — no "on time" or "late," nothing to predict. We will simply hand it two raw numbers for each assignment, `time_spent_minutes` and `difficulty`, and let *k-means clustering* find its own groupings, the same way it might group retail customers by purchasing habits.

```
In [9]: # A small unsupervised learning example: clustering assignments with no labels at a
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Notice: no "label" column is used here at all -- just raw features
cluster_features = hw_df[["time_spent_minutes", "difficulty"]]

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
hw_df["cluster"] = kmeans.fit_predict(cluster_features)

# Visualize the groups the algorithm found entirely on its own
plt.scatter(hw_df["time_spent_minutes"], hw_df["difficulty"], c=hw_df["cluster"], c
plt.xlabel("Time Spent (minutes)")
plt.ylabel("Difficulty")
plt.title("Groups Found by Unsupervised Learning (No Labels Given)")
plt.show()
```



Observation(s):

- We never told the algorithm what a "group" should mean — it discovered the clusters purely from how the two numbers were shaped, the same way it might uncover customer segments or flag a fraudulent transaction.
- Compare this to the supervised example above: there, we supplied the correct answers and asked the model to match them. Here, there were no correct answers to match — only patterns to discover.

7. Neural Networks

A neural network is a sophisticated machine learning architecture inspired by the biological structure and functioning of the human brain. It is the foundational technology behind many of the most advanced artificial intelligence applications today, including sophisticated image recognition, natural language processing, and autonomous systems.

To understand a neural network, it is essential to examine its architecture. The system is composed of artificial "neurons" or nodes, which are mathematically programmed functions organized into distinct, interconnected layers:

1. **The Input Layer:** This is where the network ingests the initial raw data. If the model is designed to analyze an image, the input nodes might represent the individual numerical

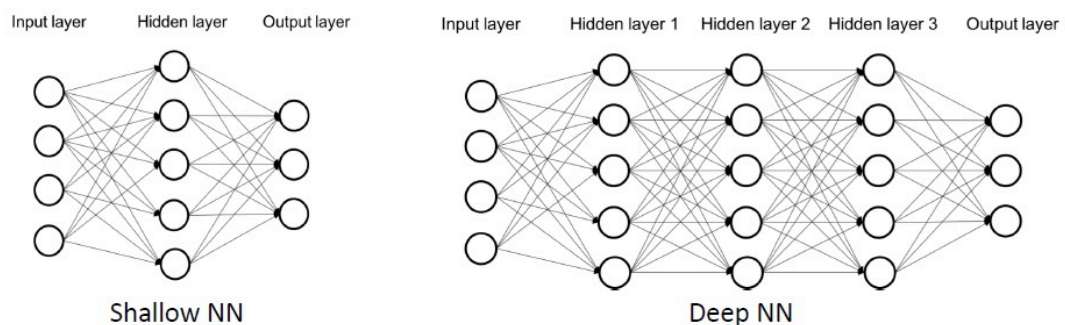
values of the image's pixels.

2. **The Hidden Layers:** These are the computational layers situated between the input and output. A network may possess a single hidden layer or dozens of them. As data passes through these layers, the network performs complex mathematical transformations, extracting increasingly abstract features. For instance, in facial recognition, the first hidden layer might detect simple contrasts and edges, while deeper layers might recognize complex composite shapes, such as a localized pattern representing an eye.
3. **The Output Layer:** This is the final stage of the architecture, which produces the model's ultimate prediction or classification based on the computations executed within the hidden layers.

The connections between these artificial neurons are the core mechanism of the learning process. Each connection is assigned a numerical parameter known as a "weight," which determines the importance or strength of that specific pathway.

When a neural network undergoes training—often utilizing the supervised learning methodologies we established previously—it continuously adjusts these weights. If the network outputs an incorrect prediction, a mathematical algorithm calculates the margin of error. The system then works backward through the network (a process formally known as *backpropagation*) to incrementally fine-tune the weights. Through repeated exposure to vast amounts of training data, the network iteratively optimizes these connections to minimize its error rate and maximize accuracy.

Furthermore, when a neural network contains multiple hidden layers, it is classified as a "deep" neural network. This specific architecture is the origin of the term **deep learning**. Deep learning models are exceptionally powerful because they possess the capacity to automatically discover and extract the necessary features from raw data without requiring a human programmer to manually define them.



8. Common Misconceptions

Computer systems don't and will never **understand** like a human (even if it is simulated, it not a human). Computer systems recognize patterns, not meaning; the same goes with **science**.

Machine learning never happens instantly; it requires large amounts of data. More data does not always mean better results. All data has bias and data quality issues to overcome.

9. Exit Ticket

1. What role does data play in artificial intelligence?
2. What is an algorithm? What is the main difference between:
 - traditional programming and machine learning?
 - supervised and unsupervised learning?
3. What is **deep learning** ?
4. What is a neural network?

10. Vocabulary

- **Machine Learning** — A branch of AI where computers learn patterns from data instead of being explicitly programmed with fixed rules.
- **Data** — Digital information collected from the world, such as text, images, temperatures, or videos, which AI systems use to learn patterns.
- **Algorithm** — A step-by-step set of instructions or procedures used by a computer to solve a problem or complete a task.
- **Supervised Learning** — A type of machine learning where the computer learns from examples that already include the correct answers, called labels.
- **Label** — The correct answer or category attached to training data in supervised learning, such as "Spam" or "Not Spam."
- **Model** — A mathematical system created by a learning algorithm that maps input data to predictions or outputs.
- **Neural Network** — A machine learning system inspired by the structure of the brain that uses layers of mathematical operations to recognize complex patterns in data.